

as specified by the `-out` parameter. To parse XML files that conform to the specification given in *catalog.xsd*, we will need to create a Java project and include the generated JAR file as a supporting library. Please note that we will need to include the XMLBeans libraries in the project.

Input XML

Before taking a look at the sample application, let us look at a snippet of the input XML. For brevity, the XML is shown with a single record:

```
<?xml version="1.0"?>
<catalog>
  <book id="bk101">
    <author>Gambardella, Matthew</author>
    <title>XML Developer's Guide</title>
    <genre>Computer</genre>
    <price>44.95</price>
    <publish_date>2000-10-01</publish_date>
    <description>An in-depth look at creating applications
with XML.</description>
  </book>
</catalog>
```

Sample application

Let us now look at an application to read the sample XML. For the sake of brevity, all attributes of the book have not been listed and/or added.

While creating the JAR file from the XSD, XMLBeans encapsulates the root XML element inside a ‘document’ object. In this case, the name of the object is ‘CatalogDocument’. We need to load this object using the factory class. Once the object has been loaded, we can navigate and manipulate it as we would a typical Java object. An annotated sample for displaying elements from the input XML, as well as adding one record, is shown below.

This is a package book sample:

```
import java.io.File;
import java.io.IOException;
import org.apache.xmlbeans.XmlException;
import noNamespace.CatalogDocument;
import noNamespace.Catalog;
import noNamespace.CatalogDocument.Catalog.Book;

public class BookSample {
    public static void main(String[] args) {
        CatalogDocument bookDoc;
        try {
            // read the XML using the factory class
            bookDoc = CatalogDocument.Factory.parse(new
File(args[0]));
            // get the catalog element
```

```
Catalog catalog = bookDoc.getCatalog();
// get books from the catalog
Book[] books = catalog.getBookArray();
// iterate over the books
for ( Book bk : books ) {
    System.out.println("Id: " + bk.getId() + ", author: "
+ bk.getAuthor() + ", title: " + bk.getTitle());
}

// Create a new book by adding it to the catalog
Book newBk = catalog.addNewBook();
// populate the fields
newBk.setId("new1");
newBk.setAuthor("temp");
newBk.setTitle("temp");
// save the updated catalog
bookDoc.save(new File(args[1]));
} catch (XmlException | IOException e) {
    e.printStackTrace();
}
}
```

You may have noted that the class *CatalogDocument* belongs to the *noNamespace* namespace. As the input XML does not use an explicit namespace, XMLBeans uses the default namespace *noNamespace* and puts the created objects in it. If the input XML contains a defined namespace, XMLBeans will name the Java packages accordingly.

By using Apache XMLBeans, the task of XML input parsing became a breeze. After generating the XSD from the sample XMLs and the JAR from the XSD, reading the input XML became a simple task and we could concentrate on the task of fixing bugs in the business logic. In other words, it was business as usual, in the context of a software development project, once we overcame the input XML hurdle. It was so easy to parse XMLs in Phase Two – an area of intense headaches in Phase One – that even 10 years later, I have continued to use XMLBeans in most projects. 

References

- [1] Welcome to XMLBeans, <https://xmlbeans.apache.org/>
- [2] Sample XML File (books.xml), [https://msdn.microsoft.com/en-us/library/ms762271\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/ms762271(v=vs.85).aspx)
- [3] Create XML Schema from an XML Document, <http://xmlgrid.net/xml2xsd.html>

By: Bipin Patwardhan

The author is a senior technical architect at Capgemini India. He believes in the value of open source and has developed many customer solutions using open source Java libraries like Apache XMLBeans.